

Active InferAnt Stream 009.1 ~ How to Accelerate?

Active Inference Mountain Car in RxInfer.jl

Source: [YouTube](#) Playlist: Active InferAnt Stream Duration: 1:30:10 |

Uploaded: Unknown Transcript method: auto_caption

all right it's December 26 2024 this is Daniel Friedman it's active inference stream 9.1 how to accelerate active inference Mountain car in RX INF fur. JL and hope everyone is having a good End of season looking forward to different updates different live chats see what happens along the way I'll start the stream off with making a GitHub push all right while that's happening I'm going to head over into cursor this is what we're going to be exploring covering today which is modifying an RX and fur package example made by the developers and moving it from a notebook format into a script format adding some more visualizations and analyses and then doing uh setup script hopefully to make it easier to get it going on your computer and meta analysis and right now there's a big meta analysis happening but we'll look at some past runs that finished just a few seconds ago okay so check the GitHub this is uh B based on the current version 3.8 of reactive Bas RX inf. JL and just to keep it separate and clear made a fork or a branch from the exact version today and then put all of the work that I'm about to go into into this examples folder SL Mountain car so it's a branch on RX and fjl and in the video description are all the links okay so GitHub push for now to start another Standalone and meta analysis so here's where I've gotten to after many speed bumps and and fun along the way this is incursor right click open an integrated terminal first if you haven't run script Julia setup. JL probably also other settings will need to be bumped up against on different settings uh also I brought it up synced it to the current version um just after after building part of it so there's probably a lot of inefficiencies it's of course Very cursor SL clawses so there could be a lot of redundancy a lot of ways to go um but I'm going to run the setup script then we'll look at the Standalone and The Meta analysis okay so this is as mentioned branches off of the RX and fur example active inference Mountain C okay let's just briefly look at setup see what's happening while it's finishing this there could probably a lot of sorting of of this um different packages are installed the RX and fur Keystone package is installed from the local GitHub repo so probably it would take some other different way to

export functions in a different way um calls probably some set of automatic dependency installations uh pre-existing explicit calls that I uh typed in like most recently this stats plots so definitely figure out what else could be changed to get this working on your own computer but at the end I hope it accurately says yes RX infers functional some of the core packages that we need are loaded here's the Julia version project path RX infer path and then how to run the scripts and and that can be probably developed okay so that's the setup then there's the Standalone Julia Mountain car Standalone this one has today's date and a file name just to give it a static version as as a single file snapshot but the meta analysis is much more scattered across methods scripts so the single Mountain car stand alone has the date Okay so let's look before we go into the code at how they set up and characterize the mountain car problem okay and so it's written in this sort of notebook blog post multimedia style active inference Mountain car this is from the RX and for JL documentation a group of friends is going to a camping site that is located on the biggest mountain in the Netherlands they use an electric car for the trip when they are almost there the car's battery is almost empty and is therefore limiting the engine Force Unfortunately they are in the middle of a valley and don't have enough power to reach the camping site night is falling and they still need to reach the top of the mountain as Rescuers let us develop an active inference AI agent that can get them up the hill with limited engine power okay before we go into the technical details here's just a single perplexity prompt give an overview of the mountain car problem from control theory at four level SL types of depth plain archetypal Ultra Technical and esoteric and also generated some images so we can generate some more images now or people can suggest other prompt so plain understanding the problem involves an underpowered car stuck in a valley between two Hills the car's engine is too weak to climb directly up to the right Hill to reach a goal flat instead it must swing back and forth building up momentum by driving up the opposite Hill first until it gains enough energy to reach the target archetypal perspective this scenario exemplifies the fundamental principle that direct paths aren't always optimal the car must first move away from its goal driving left to ultimately reach it on the right a counterintuitive strategy that demonstrates how complex systems often require indirect act approaches the problem has become a classic test case for reinforcement learning algorithms because it requires learning this nonobvious solution so the active inference approach that RX Ander is going to implement doesn't have a trained reward or proposed reward function on some data set of successful or not successful traces it's going to be directly from the distributions okay technical analysis we'll see in more specifics getting into the scripts and then I'll be the last one

esoteric implications the mountain car problem transcends its mechanical nature to represent a metaphor for optimization in complex Landscapes the system demonstrates the necessity of building potential energy through oscillatory Behavior the interplay between kinetic and potential energy transformations the emergence of sophisticated behavior from simple physical rules the relationship between local Optima Valley and Global Optima goal the problems enduring relevance stems from its ability to challenge both traditional control methods and modern machine learning approaches while maintaining mathematical elegance and its Simplicity okay so the goal for this stream and then we'll look more at the specifics of the mountain car and the code itself to learn about RX and fur hopefully correctly or at least helpfully implement one useful experiment to learn about Julia and all these different tools and shout out to Andrew for the early Julia adoption and and insights um here is using cursor 44.8 RX andur 3.8 or two Julia 1.11.2 and then for the llm in cursor Claude 3.5 Sonet 2024 10 22 all right here's the finished single Standalone that we ran started during the stream it is going to implement two different simulations with the same physics and two different approaches to strategy first is going to be the naive approach which always pushes to the right and then the second is going to be this active inference approach with a given look ahead period and this script goes into the outputs so here's the new one in green here are the outputs of the single simulation and this will give an intuition for what the single um Standalone does so first comparison dog GI so here the single parameter combination that is used was chosen it's I think similar proportional or the same to what was used in the original example notebook and there's two different dots representing the cars position simulation on this landscape and we'll look at how physics is given on this landscape so the rules of physics are specified and the elevation and the potential and the kinetic energies are calculated in a sort of basic physics simulation style not simply uh modifying some attribute of the direct landscape function itself and they both start at .5 and they're both trying to get to positive point5 the colors will change I think throughout but naive control is red and it drives right and it oscillates a little bit but reaches a fixed point where the friction and the gravity are equivalent to the engine's limiting force and that reaches this staple point the blue active inference agent has this anticipatory simulated trace and I think it's it's definitely worthy of another long investigation or how does it how does it distributionally fit this policy anticipation and what is the computational complexity as we scale the time Horizon of inference but we can see that with this preference for being higher up the active inference agent can see that it can get higher up to the left than it is on the right by moving that way and that from uh already being up on the right slope and

oscillate its way to the goal position and you can imagine that that there's going to be um combinations of physics and engine force and goal position that are easily or impossible to attain right going through the rest of the images in this single case um output but first going to start the well let's just go through all these first okay so this is the action distribution the naive control has is always picking that one value to go right with one whereas the octave inference now it's in tan it or red has at least I don't know if I'm perfectly implementing what the notebook does but at least it does have some policy variance and it one that for this parameter combination does get it up to the goal action comparison so there's another way to look at that this is the knife control with now in blue with the yal 1 policy function output active inference agent having this more back and forth control policy swinging between two and -2 I'm not sure what this if this is scaled to a different number it's but none of this is meant to say that this is the best fit it's really just a first step towards figuring out what what kinds of tools and interfaces for generative models and for RX and fur can we use to start doing some useful parameter sweeps across different regions of of parameter space and structure models okay these are State action Maps so here's position on the x-axis velocity on the Y AIS for um there's another one that makes sense yeah so here's here's the trace of the naive n blue and active inference showing these different trajectories they start out with zero velocity at .5 here's a version where the vectors added the naive control follows this blue spiral to its fix Point attractor the active inference agent has this oscillatory Behavior reflected with a loop not just a spiral that ends up reaching a higher net position via increased velocity which is what it needs to get up the hill here's the predictive element of the active inference agent here's velocity distribution there's a wider range of velocities ranging from the the negative to the positive positive further in the active inference agent um energy comparison this is not like variational free energy this is the kinetic minus the potential or kinetic and the potential energy so we can look at this calculation though and Energy Efficiency I think is the integral or something of this but oh not exactly sure okay so this state action map is showing that trajectory and then mapping overlay with which action was taken so starting here 2.0 was used that's moving to the right and then abruptly the policy switches to -2.0 then policy switches here and then the position is um approached and I I think that's an interesting question like to what extent does it have inertia to meet the goal or does it engage in a sort of zenos paradox slowing down as it converges if it has the engine control so it has to be kind of like a run past the Finish Line situation at least in terms of the explicit preference distribution otherwise you you get sort of slowing Behavior leading up to the go I don't know that could just be a total

artifact of this hill okay that's the that's the uh single Standalone output okay now config do is going to specify a metaanalysis sweep so here's what what it's going to look like here it was 200 simulations so times definitely interesting to profile how much is the startup time for one app model and then how what is there any other further caching because starting the first one seems to take longer at has the sort of RX and firm Machinery comes online I guess but we we'll look at the code viz too it should be interesting um and and again it's all just initial takes for hopefully people and others to improve on it so here's the config dot metaanalysis so there's two knobs in this parameter grid sweep the force limit of the engine and the friction coefficient of the landscape the gravity constant is kept constant for each of the engine force and the friction coefficient you give a minimum and a maximum so I think the one that just ran maybe it was this one I don't know oh let's see [Music] 77 we'll just run a clean so minimum and maximum force and friction let's do a higher resolution 1 to 27 01 to 27 so this is going to do $15 * 15$ multiplied by the number of episodes per parameter combination so that can be used for technical replicates so then that parameter combination we can understand the variance profile of it too I'll I'll set that to two maximum steps per episode let's leave it at 100 planning Horizon leave it at 40 initial position this is also not position and velocity third would be acceleration etc etc etc snap crackle pop all that um div grad curl Target state is the position and velocity preferences so I think again interesting to see are those convergent or does Nera carry some plotting here kind of as an experiment kind of as a bit of a um local coding Optima uh some of the plots are are in in asky plain text and then some output so 155 so that was just to sort of recap run setup. JL confirm that RX infer works then run just the stand alone Mountain car Standalone 12264 and that will do the simulation we did that close that window this is another thing alone now clearing now we're going to run the Met analysis so hopefully this will work meanwhile let us explore the code VI briefly visualization so not sure if this is accurate this is uh TBD with RX infer experts but here the prompt for this um graphical visualization was I'll I'll modify the prompt just so we have three free flows left for this out of five here's the meta analysis let's close that one meta analysis plugging away on the terminal we'll see if it has to be D or rebug and then we'll do a graph V free version on the free on this flow visualization emphasize the component of active inference and the free energy princip okay looks like we passed the first phase so this loaded the configuration Tomo set up the parameter batch objects plus methods so now we'll see how the simulations go um the first part I think takes longer and then the second it hits a second rate I'm not sure which I think might maybe does the the active inference ones then all the naive and the the so we we'll see

how long it takes to click to % and then see how long it takes to finish okay meanwhile here was the graph this so the The Prompt was and again I I I hope slash don't know this is accurate would be really interested to explore how we could develop these methods and get some um stable versioning if these are accurate and uh use them for learning RX invert so Trace how a probabilistic model specification transforms transforms from App model macro definition through node creation it's to final compiled graph structure including node contraction optimization emphasize the component of active inference and the free energy principle not sure that matter so here is ostensibly looking through parts of this package not sure how much of a rocket and all these other pieces are in there but you know here's some references in the code again I'm working in the RX infer Branch so here it does find a place in documents so the app model macro is passed to graph PPL package here's a section this is kind of like using cursor SL Cod VI as a rag for code and docs facilitated because the RX infer documentation is is on their GitHub as well so then let's see app model macro definition uses graph PPL and that gets customized by reactive message passing graph PPL back end okay maybe transformation pipeline compos simple operators folds mathematical expressions and adds brackets to ensure correct argument handling so the app model parameters and syntax get kind of unpacked inject Tilda right hand side aliases that creates nodes and variables based on model specification from the nodes so this is the statistical base graph I guess interesting although some nodes in RX INF fur. JL already come with their own meta structure users have the flexibility to Define different meta structures for those nodes and also for custom ones a meta structure is created by using the struct statement in Julia okay that is converted to a factor graph that's sort of the post 2017 is work with bird def rise friston par here's the free energy optimization component okay so the factor graph construction I guess whether in the scheduled forny classic non-reactive style or ostensibly but not necessarily in the RX infer reactive message passing setting probably the message passing can be scheduled or reactive differently here's variational free energy variational free energy then defines a functional objective that includes the model and a variational distribution over the latent variables not sure if this is going to always be the case for any graph or how how how that um comes to be for a given Factor graph but these are just questions to to learn from Explorer and then here's a specific bet free energy I know Dave told me it was one way or the other but probably I said the wrong one but the Betha approximation um okay meanwhile 21% done with this 225 time something um metaanalysis sweep the BFE can be iteratively optimized for each individual variational distribution in turn optimization of the BFE is thus more manageable than direct optimization of the vfe so they use also these

methods that that in the RX infer group this year we we kind of talk about as the what are they called the six functions so they are a convention which provides interpretability and structuring for ARX And action perception Loops by using a canonical set of methods so the and thanks to many of the participants throughout this year in getting the documentation doing all these experiments too so I I I I don't know how accurately I've implemented it but here's what we had and there's different ways to do it the so in the structured scheduled single time step all every node one time step Factor graph Logistics um it's not reactive it's like moving the whole simulation ahead one click the the I mean what I understand of with the reactive message passing situation is that it brings computational reactive methods to the local variant of a free energy the be free energy that enable variational inference to be done in a in a flexible and effective way okay node contraction oh the six function so once into the reactive setting whereas in the scheduled version it was just iterate through this list of of um operations let's do divide this is graphical brain belief propagation and active inference forist and Par devise 2017 figure one showing how one time step updating PDP graph up here translates to these analytical updates occurring in this scheduled order on this forny graph which is a transformation of kind of nodes into edges and vice versa between the bay and the forny graph and there's also recent other papers and live streams and such but this 2017 is just laying out the relationship between the single time step of basy and updating and the update rules that can be scheduled on that for graph so then reactive moved that into a much more General situation where different processes can be playing out with different Logics and speeds so there could be like One sensor at 100 htz and one at 1 Hertz and you're neither sampling too much or too little however then there's this question of okay well we're going to specify the app model using the RX and for app model syntax looking like this here's transition function modeling transition due to gravity and friction so updating position and velocity given physics the the structure of affordance or policy in action space um inverse Engine Force not sure why it needs to be defined here this is I I hope just putting out some stabs and phone trails in into the dark for people to clarify perform techno invective make it clearer make other documentation all all these kinds of things I'm I'm and I'm I'm not sure if I've completely perfectly mapped it into the script form but from this example this is the proper reference version the app model function for the agent type like Mountain car is defined like in this generative model specification language where for time steps during the time steps of the simulation there's a direct distributional updating occurring so that's just defines all of the updates to occur one could imagine that that that doing the order differently would matter so

maybe it could be as simple as just these happening but the the order could matter and you might not want the order to matter in exactly the same um order as the code are lined but rather to take this um all at once rep representation and then further just from a package perspective it's desired to have a more of a flexibility with how you call the statistical distribution so that again those different Logics can be hooked up to what is called otherwise if it's only this expressive and none of those further steps were taken then this could just do something as simple as iterate through in this order but then to put in logics well like actually do this one twice as fast or something like that it would be needing the kind of rewrite that would result in some of these steps um so technically the agent here goes through three phases act execute observe that's one step infer and slide kind of like act infer surf act execute observe in this step the agent performs an action onto the environment at time T and gets another gets an observation in exchange these observations are basically the prediction of the agent on how the environment evolves over the next T time step so act and get an updated position momentum infer after receiving observations the agent starts updating its internal probabilistic model by doing inference it finds the posterior distributions over the state and the control distributions three slide after updating its internal belief the agent moves to the next time step and uses the inferred action from the previous time step to interact with the environment and that is where you pick back up with one so this is the kind of UDA toe inactive Etc and these steps are kind of like midlevel functions um explored on this page here with six where there's a future or here it says the O and some of this might be from an earlier slightly different version it's just all sort of directionally uh in the ballpark that that because the graph doesn't have this intrinsic scheduling to it then and and you can have so much flexibility with a custom scheduling um custom nodes message rules all that kind of stuff that um you need to bring how the graph is called and updated and operated more explicitly so that's a good thing it's just that it's still slightly evolving with many of the package semantics and examples and norms and uh there's probably many patterns to explore and not necessarily some sequence or repertoire of these midlevel possibly semantic or or or sub agentic like steps that give a lot of nuance with describing is it o o d a or is it a d o o or is it o o d o o d o d Etc okay coming back to see how simulation okay 52% done on the meta analysis so that was um that was the code Vis let's do another code VI help us totally understand how the stand alone do mountain car let's just see if this works help us totally understand how to Standalone car script calls every minute particular and Etc please thank you 55% done reindex then we'll look we'll see this code Biz then we'll look at the um output of the meta analysis okay but briefly just to look

over what they get to so they describe the analytical process formalization for the mountain so again it's not a proposed reward or utility function or anything trained on data it's just a distribution that's physics grounded using just simply a gravitational force on the hill landscape depending on the position and that's implemented also using possibly some special characters and some special Julia syntax that will take some learning and some package specific um so to be learned and better understood but here's where they lie out the functions that create physics and create world and the sort of world generative process element they have a static image just to confirm that they've initialized everything here's the naive approach this is the naive the I think I have implemented it slightly differently with just having it all output a one each time step maybe they're pre-filling it as well and then here's the visualization of just the naive policy having that initial ramp up then fixed Point here's the active inference generative model okay and we'll read now let's help them solve the problem with an active inference approach particularly we create an agent that predicts the future car position as well as the best possible actions in a probabilistic manner so not from data not from data sets we start by specifying a probabilistic model for the agent that describes the agent's internal beliefs over the external Dynamics over the environment the generative model is defined as follows so here are the equations and in the GitHub for basically the variables in the base graph and I I would look forward to um using some of these new visualization methods to to visualize the factors and the app model from The Notebook visualize the Standalone and The Meta analysis to confirm that the um model is being analyzed correctly okay now let's look at Standalone well let's just finish with the what they have in the documentary here's the um variables as defined here's the key RX infer app macro app model macro and meta blocks are used to define the probabilistic model and the approximation methods for the nonlinear State transition functions respectively in addition the beliefs over the future States up to T steps ahead of the agent is included so this is like the agent class equivalent which we looked at briefly before defined as those distributions over those variables after specifying the generative model let's create an active inference agent for the car technically the agent goes through three phases so here again are those midlevel sem IC bundles of sort of distributional updating cognitive sub agentic process Etc that are being projected into this three cycle act execute observe in fur slide but that's again as far as I understood from kis and others and and learning this year that's just a convention and other orders or other kind of um and that could be a good place to play with this simulation um and again to confirm through the flow of the simulation with the code viz how the notebook the

Standalone and The Meta analysis um might be correctly or incorrectly applied so in So Below we create the agent through the create agent function which includes compute act slide and future so here's future kind of but it was mentioned above here is where the reactive message passing component is called not exactly sure what they made about the message so again just a learning artifact to try to understand the the extremely interesting potential and directions we can go with the kind of flexibility that we have with RX and fur and how cool their Direction and vision for the open source element is too so here's the create agents method now it's time to see remember now it's time to see if we can help our friends arrive at the camping site by midnight question mark here is the Julia syntax there's a combination of initialization methods parameterization agent creation further parameter specification step through experimental protocol then this discrete time Loop for the number of time steps as the active inference simulation's going to go and then here's a very concise animation method and so here it's it's a little bit different but it's the same kind of General picture that is shown here different scaling colors um but that's what is happening in outputs comparisons okay and they have more information on um way message passing plays out with this example okay all right let's look at the code viz for the whatever we prompted okay 83% done with the with the big meta analysis okay so here we go here's Mountain car Standalone 1226 2024 first loads config dotel like I don't even know if it does it does that's one it also configures a shared logging function and a visualization probably data collect ction arraying function so then these config informed visualization and logging methods are holding the traces of these two agents as they're simulated within a shared parameter configuration like physics and friction that's each cell of this meta analysis that's ongoing and each of these agent folders We could say has like a naive control agent for that physics combination and a active influence agent for that physics configuration the parameterized sort of setup for the simulation and everything loaded in place which can often be a lot of the the hardest part sometimes cursor plus llms knocks it out of the park other times it's it's um just two steps backwards one step forward um Mountain car is called interesting Mountain car. JL even here looks like it goes to code of conduct here's the read me let's look at that on GitHub let's look at some of these methods wow though interesting okay 95% done with the um meta analysis here's here's the mountain car model method so here's Standalone stand alone's parameters are I I don't know if um terminally I'm sure it could be restructured are written at 300 lines in just wanted to start there to highlight that so here's how I understand what's happening in the Standalone it's defining this as the mountain car module and and I think a a better package reference and function reference could

go a long way here at the package and at the specific method level RX and fur and reactive message passing sub kind of second level Consultants are called and some of the other packages that are needed visualization. JL is brought in for some methods also this file exports public functions so other scripts can call include on this script because and and get these functions here are some all right here we go completed 900 simulations with 80 successes okay good so that that means that some of the simulations should have worked some of them didn't work so at least we picked some interesting area not sure if it's going to visualize everything properly okay here's the app model function with like the same I hope um distributions and such here's the create physics here's the create worlds here's the create agents here's calculating the kinetic potential and total energy so here's the the potential energy with 9.81 gravitational constant okay no errors from the metaanalysis but we'll see what it ends up outputting what was output so here's the physics-based energy calculation not to be confused only to be distinguished from the fact that the message passing updates are doing a free energy minimization but this is referring to the physics I don't know what it's if there's a just the physics phas base for the Newtonian mechanics here's the method to call for the animations that's the module then here is when that Standalone is called by itself it calls in all these other packages here we're at three line 300 here are the parameterizations these May override or vice versa and set the shared parameters for the naive and the active inference approach here's the act the naive approach here's the execution of the approach this is sort of a procedural way they defined um just again like these are this is like if an example of something that might be totally Irrelevant in a given moment or it might make a huge difference in a simulation so here they put 100 ones or n naive time steps of ones and then just accepted that Vector here possibly more error prone possibly introducing um disastrous logic there's a procedural generation of setting the action at that time step to to one and that enables some of the single logging here's the active inference approach so it's pre um figured with the AI namespace subscript here is the compute Act slide future agents not sure if this 04 gets overridden we'll see maybe in the meta analysis here's the active inference simulation in a let block which helps with the I I I don't mean impenetrability security but in the error tolerance and for allowances in program so different sort of footholds entry points for logging and an understanding and for a given situation maybe more or less there's a lot I I I don't even know so many different ways to write different codes so here's the observation of the position and the velocity from the current states modalities factors here that's the initial position here is the time step discrete time sequence so this is kind of like there's the topos of the the topology of the action

perception partition and then this is like the chryos possibly also Kronos for specifying how those different topological aspects of the action perception partition get enacted sequentially in this case but with the expressivity to have other Logics underneath these uh just current transient Norm specifications so set the action to the ACT function push it onto the action trace and execute the action then observe push the observations store the predictions and and the future call check against the target update beliefs so maybe that's the sort of running through the Finish Line component like it checks whether it's actually at the goal then it updates its beliefs or but I'm not sure and logs every 20 time steps here's the calculation of the energies the visualization on the analysis of the naive and the AI comparison plots outputs some summaries okay so that again goes to the outputs folder now let's look at so 17 okay so here's an anova but here's some earlier ones that have many more let's try to get these back first just going to explicitly bring in the uh meta analysis and the mountain car methods okay please ensure that all asky art and non asky parenthesis quote normal plots are fly output by The Meta analysis then we'll update the toml just to do a tiny one just to verify that so we'll do 2 by two so this is kind of a common sort of micro Dev Loop situation with of course just breaking unworking things but visualizations sort of trickling off needing to be fixed and then the sort of trade-off between pursuing a local branch and then pursuing a deeper Branch so for example here was using the asky text Art so this was on 400 50 for each sweeping so let's see these were the plain texts we'll see if we can recover these but you know hilarious if we can hilarious if we can okay here's just a sort of summary on the overall success versus failure of naive versus active inference so TBD if it's exactly perfectly doing what the Standalone does and again if that's exactly the right or only way way to do it in the example but at least in this parameter Zone active inference is doing better so there's at least some cases where there's a better policy than just going to the right success rates so here's sort of an asky art uh visualization not sure if the dimensionality is right away but it's sort of it's um example here's the difference here's some different plots this is why it was a local opma because like it looks kind of sick so I thought okay it could definitely be also improved but it's also like it's cool it can stay parameter analysis so here's success rate versus force in the naive success rate not sure why this wouldn't be a monotonic distribution different plots like plotting this by that plotting this by that okay meanwhile I'm going to accept blindly the edits that it made to the metaanalysis double check the toml we're doing much smaller situation clear run it again back to the earlier outputs heat Maps is empty so that was one of the non asky art energy analysis so this again suggestive that the simulation really is happening which is kind of the

question of these different cases but it's limited ability with the aski so here's um another visualization that's it's like the control effort distribution by agent type almost always is at 100 but then there's a few other ones so not sure if those are spous or how those come to be in the naive control effort case but here is this um different distribution okay and Analysis is empty so here's an example of the sort of extended startup time of RX infer okay so this will probably have to be resolved but here is setting up the smaller simulation so it sets up the whole batch whether it has to do two or 200,000 and it's going to go a lot faster it's already at 12% okay okay so um I'll just sort of read some of these second uh closing Parts while we finish up this meta analysis if you're watching live though write any comments or questions in the chat okay so here's some open source to do um at the package scale just verify and improve you know anyone who wants to try can clone the repo get the setup. JL working on another setup on the statistical front we could do better statistical distri description work visualization and Analysis within and across simulations so sort of distinguishing and building some statistical power apparatus for technical replicates and biological replicates or experimental conditions 19% on the simulation um on the formal and the analytical structural side well one approach is from the initial RX and fer teams notebook and from the Standalone and from the uh more meta analysis style apply uh generalized notation notation GNN style which we we we can actually try this in in this uh stream briefly so we'll leave that one up um category Theory here added a couple of links before so still analyzing The Meta analysis at least that's a good sign um so wanted to highlight two recent blog posts from August and November of this year by colleagues at The topos Institute so the earlier one is a blog post by Sophie libkin and David SPAC a polinomial account of basan update so here they describe this poly tree notation this is a very math Arty useful uh overview for the Paulie category theoretic notation and some operations and probably a ton more for basian update and and later they did work including more of the action side and then here's a post from uh November of this year by David SPAC describing a paper uh Dynamic task delegation for hierarchical agents okay met analysis finished with an error no look just fix it thoughtfully so neural wiring diagrams for message passing in multiscale organizations sounds to connect on a lot of ACM topics so they wrote in our recent paper Dynamic task delegation for hierarchical agents Sophie Lipkin deny described a task delegation from an agent to a team of subordinates as morphisms in a certain operad called org M however such morphisms incl include a great deal of data and the examples we gave there barely scratch the surface of what's possible in this post I'll show how recumbent neural wiring diagrams with arbitrary feedback and quote axon splitting give rise to

a message passing language for describing certain such task delegations that is I'll Define a cyclic operad R nwd and a function RN WD Arrow orgm right now what are you doing rnw ID and there could be a lot of connections here with the message passing so I want to just highlight those see if anyone is interested to explore that direction um just in terms of making okay now back to the code here accept the change resync it clear start up the meta again um on the sort of action perception Behavioral Science component fitting data playing with different noise patterns and levels and then also figuring out some other strategies that can get plugged in other than the the naive one so just for reference this is the I believe formerly known as open AI gym now called gymnasium do.org so Mountain car here's how they describe this the environment is part of the classic control environments which contains general information about the environment so description this is how they describe it on that gymnasium situation the mountain car mdp is a deterministic mdp Markov decision process that consists of a car placed statically at the bottom of a sinusoidal valley where the only possible actions being the accelerations that can be applied to the car in either direction the goal of the mdp is to strategically accelerate the car to reach the goal State on top of the right Hill there are two versions of a mountain car domain and gymnasium one with discrete actions and one with continuous this version is the one with discrete actions first appeared in Andrew Moore's PhD thesis 1990 so here's just how they do so that that could be cool to bring the RX infer active inference agent to the gymnasium and vice versa another uh angle here look into the links go to the multi-agent page in active block FR project document and and and probably other and and better um sources for this too but this is just from from years ago in the Cyber physical case and the Ant case some of the different ways that hopefully RX environments and or similar will help enable with the expressivity to talk about a sort of pure blockchain stigma G Niche modification neighborhood where you have a kind of centralized at that scale of analysis Hub and spoke to and from like for example in the blockchain case people might talk about sending from wallet a to wallet B but from a message passing perspective it was wallet a to the blockchain and then the blockchain update changes the efficacy at a future state for B uh whereas the in this case maybe explicit it's not the best term here but spatial collisions and direct interactions are possible as well as environment interactions so okay we're back to the meta okay already it looks like more folders have come out 16 simulations with four successes so again some of these should um okay so here we are within this most recent we'll wait for this to finish and then we'll look into this most recent meta okay the heat map chaos surprise chaos Mouse over or just copy and paste it into the composer please thoughtfully address all

so that the visualizations of all kinds are safely plotted and fully logged output okay
meanwhile what is happening in these folders okay heat Maps went empty but here's what
came out before that okay so here was the smaller parameterization with these other asy so for
the for the cyber Punk solar Punk RX INF fur Enthusiast this one oh is much bigger the the uh
size of the asy art can be specified in the tel plot width and plot height okay meanwhile the
agent is chugging away over here okay then parameter analysis with these bigger ones so not
exactly sure how accurate everything but okay heat Maps empty that's kind of part of the issue
okay interesting with a naive distribution asky plots nothing okay accepting these visualization
updates clear let's just delete the ones from after 18 call The Meta analysis it's going to run it
so how can we use RX environments or Andor others to model this kind of direct
communication amongst agents low road or high road to model transient communication like
propagative radio EMF light schooling SL flocking or sort of proximate antenna antenal
interactions as well as to model communication through SpaceTime cones thinking more
about topics like Niche modification stigma G across trial learning and phermone
psychophysics in the active inference case Kus live in the dock okay so here appears to be a
key part of the issue so we'll see how how much this um interferes with it on this new run here
happening here but there's some overlap between the asky art let's just preempt ensure that we
are not quote getting it twisted with the plots and asky are art plots math art plots please check
over everything and fix and trench to be 100% functionality okay checking out the read me
structure of the repo Standalone implementation physics engine octave inference agent naive
agent visualization meta analysis framework at least the hope of measures set up with setup
script run standalone or The Meta analysis update Pro config dotl hopefully project. toml the
one with all the package stuff should be taken care of by setup but submit a request or or let's
figure out how to make it simpler and here are some of those key packages including RX and
fur okay all right finished the 16 simulations now it's plotting so we anything with 18 see but
we're already kind of we're already on another okay rerunning it starting now to to fix but
basically this is the whole coefficient slash question how fast how good how augmented how
clear how well documented how aligned how useful how interesting how novel how valuable
so let's let that simulation go okay while the meta analysis is happening let's have just a more
folder let's just see how it does with a who done first in who's on first write a please write a
comprehensive absolutely informed and farsighted slash out cited parentheses indoor outdoor
specialist status who's on first style dialogue or more about 1920s americ Baseball USA OS
specific lore and all the contents of the read me and Mountain car active inference reactive

message passing scheme as we've tried to and dot dot dot I'll help write a humorous Abbot and Costello Who's on First style dialogue that cleverly weaves together classic baseball references with the mountain car active inference implementation let me write this in the who's on first. MD file 69% done with the metaanalysis okay okay let's see if okay we'll just do the the the one shot okay pushed it to GitHub accept the changes clear run at analysis reload the repo more all right just one shot just so that we can close out who's on first Mountain car Edition say Castello I've been meaning to tell you about this fascinating new project we're working on at the Ballpark oh yeah what's that it's called Mountain car active inference a mountain car at a baseball park what are you nuts no no it's a simulation we've got two agents trying to get up the hill two agents like baseball agents no one's active inference and one's naive well that's naive of them both who's controlling them exactly who's controlling the active agent what's controlling the naive agent and I don't know is measuring the performance metrics wait a minute who's controlling the active agent yes precisely I mean the Fell's name who the agent controlling the car who's controlling the active inference agent that's what I'm asking you and I'm telling you you who is controlling the active inference agent look let me start over when you run the simulation who plots the result oh no who doesn't plot anything what plots the results I'm not asking what plots the results what does plot the results creates beautiful heat Maps too I don't care about heat Maps I want to know who's plotting no no who's controlling plotting then who is plotting no like I said what plots everything all right wise guy let me try another way the car needs to get up the hill right right so who calculates the force needed now who only handles active inference I don't know calculates the forces you don't know who calculates the forces no I don't know calculates the forces that's what I'm asking you and that's what I'm telling you I don't know calculates the forces then who does no who does active inference I don't know does forces what does plotting and tomorrow handles the parameter sweep parameter sweep you bringing in a grounds keeping now no no tomorrow is our batch simulation Handler today is not involved at all how can you run simulations if today is not involved why would you want today involved I'm not asking why oh why is busy analyzing the energy consumption I don't even know what we're talking about anymore that's the spirit now you're getting it I don't know it's a crucial part of our implementation ah hey wait I haven't told you about because of statistical significance testing yet later that day okay let me just ask one simple thing sure in the config file who sets the initial position oh no naturally sets the initial position naturally yes naturally I should have stayed in the minor leagues don't forget practice tomorrow we're running a full parameter sweep distance screaming only to

return to more airs let's see how far it got still no heat Maps Okay um I'll give one more minutes or so if anybody wants to write a question in live chat otherwise look forward to seeing what people think um see where it's incorrectly or inefficiently applying active inference with RX infer because this is a really promising direction there's a lot of interfaces for specifying simulations making uh versions of the textbook uh examples all all kinds of fun research and education and application that RX infer is enabling so I will accept this last edit make the last push yeah composer being able to edit across multiple files see this is good it looks good but it's really hard to see just from a quick scan so I'll just leave it at that cursor make last get push okay verify that closing edits were sent all right thank you everyone enjoy I wish you a good closure of the Year beginning of the next good accelerations in this continuous discreet complex scenario we're in okay thank you bye